

A decorative pattern of hexagons in red, grey, and white, some with thin lines extending from them, located in the top right corner of the slide.

ROS2 Topic通讯

构建机器人的神经系统



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



目录

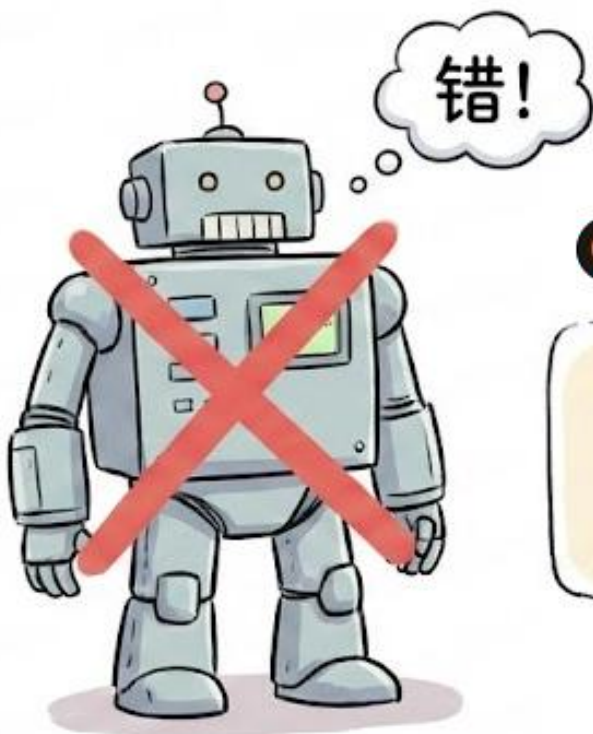
Contents

- ◆ 直觉与工具
- ◆ 标准开发实战
- ◆ 架构进阶
- ◆ 综合闭环应用



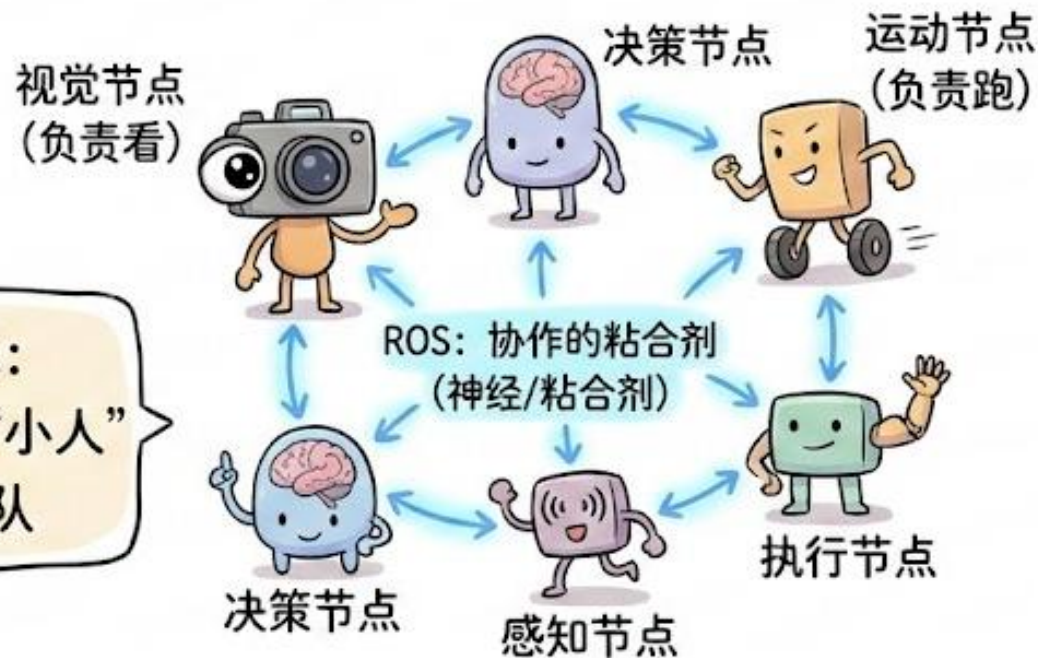
直觉与工具

打破固有印象：机器人不是一个人，而是一个团队



固有印象：巨大的单一程序

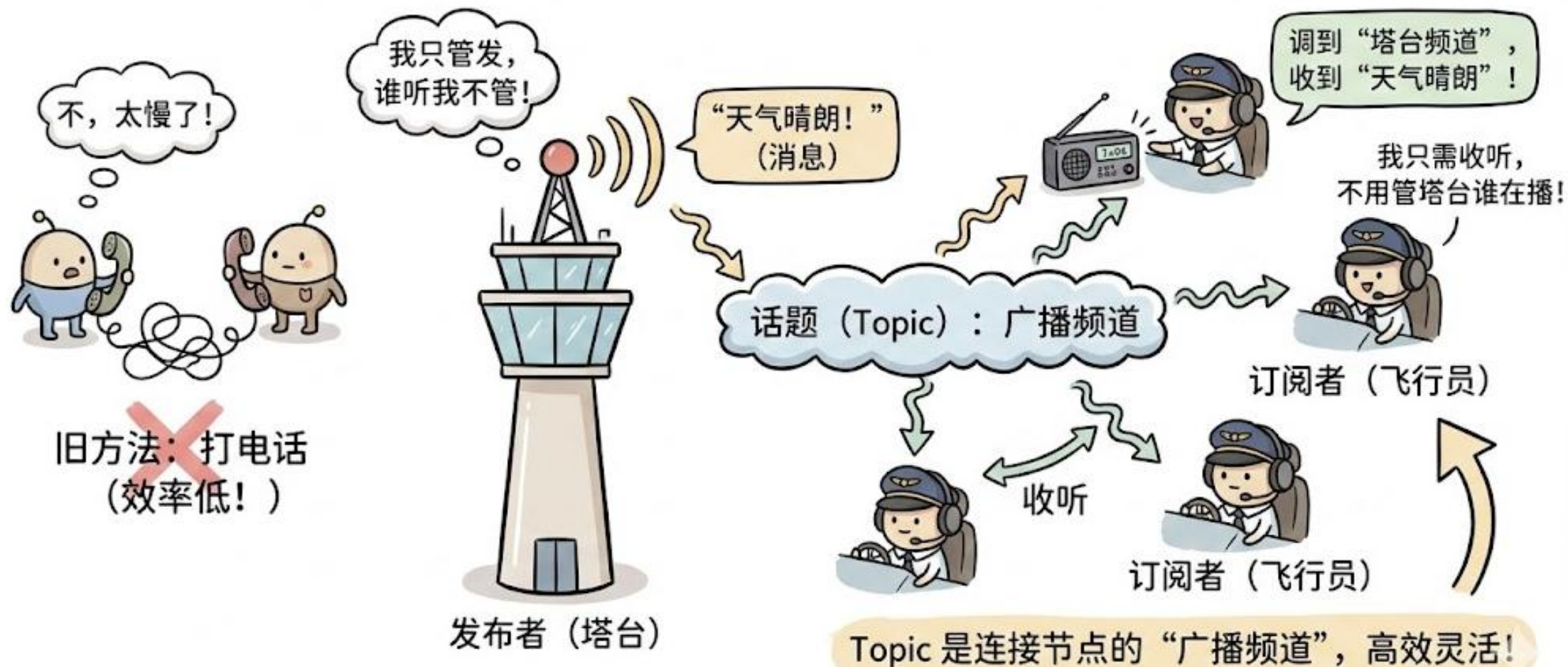
ROS 的世界观：
由无数独立的“小人”
(Node) 组成团队



工程师理解：微服务架构

机器人是一个由无数独立节点协作的团队。

ROS 小人交流秘籍：发布/订阅模型 (Pub/Sub)



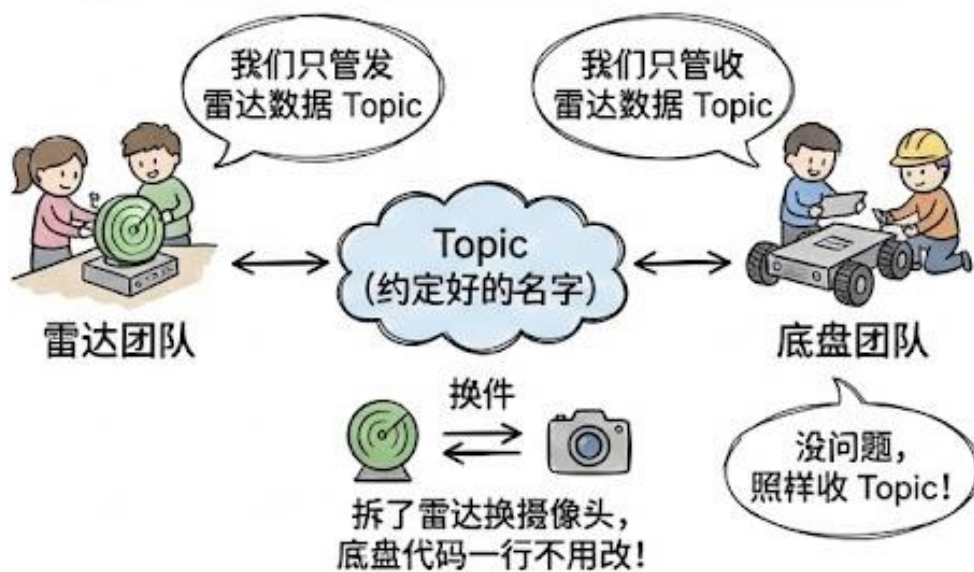
工业界为什么非要用 Pub/Sub?

核心词一：异步 (Asynchronous)



发送完就走，无需等待回复。效率高！

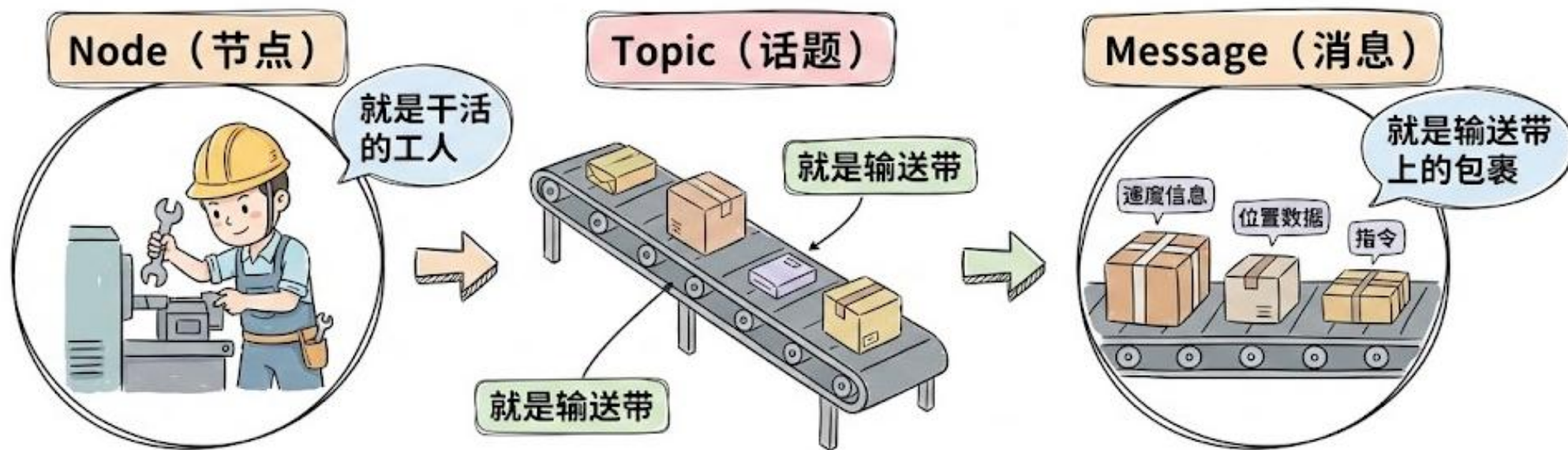
核心词二：解耦 (Decoupling)



系统设计的最高境界：互不认识，灵活扩展。

这就是 ROS 能支撑起数百万行代码复杂机器人项目的秘密！

ROS 核心概念“行话”统一



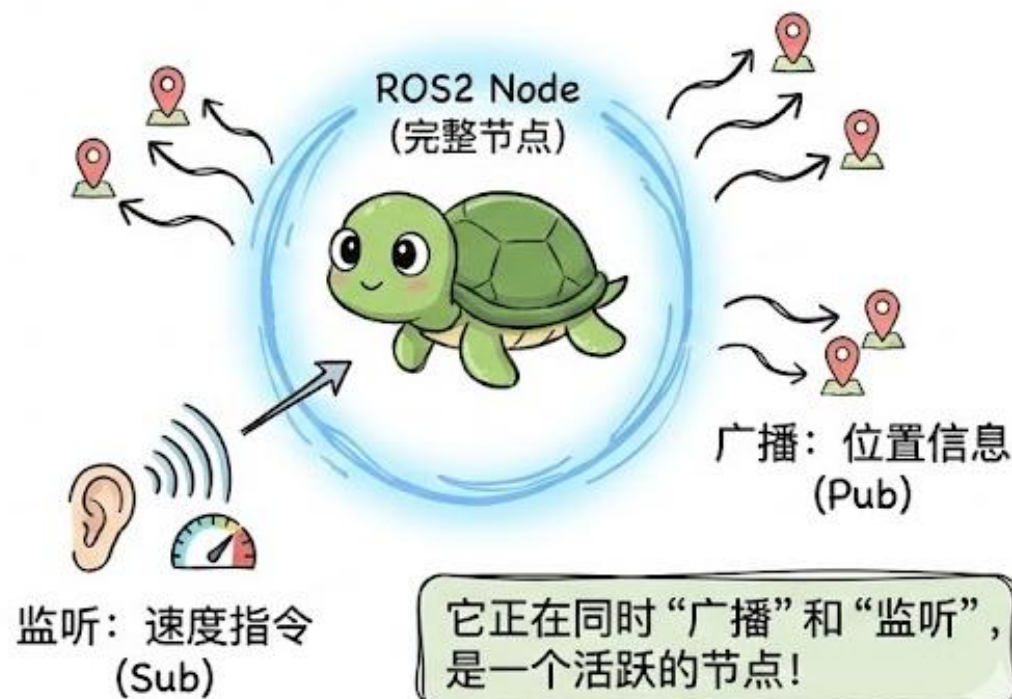
这三个概念将贯穿我们整个职业生涯。

今天的实验主角：呆萌小海龟 Turtlesim

启动 Turtlesim



别小看它：一个完整的 ROS2 Node



ROS2 系统诊断：像医生一样“看病”

第一步：“查户口” (ros2 topic list)



ROS医生

```
$ ros2 topic list
```

```
/rosout  
/parameter_events  
/turtle1  
/turtle1/pose  
...  
...
```

看看现在有哪些
频道在广播
(Topics)

第二步：用“听诊器” (ros2 topic echo)



```
$ ros2 topic echo /turtle1/pose
```

/turtle1/pose
(数据流)

海龟在实时汇报
它的坐标，拥有上
帝视角!



实时数据流
(上帝视角)

记住，看到数据流，你就拥有了上帝视角。

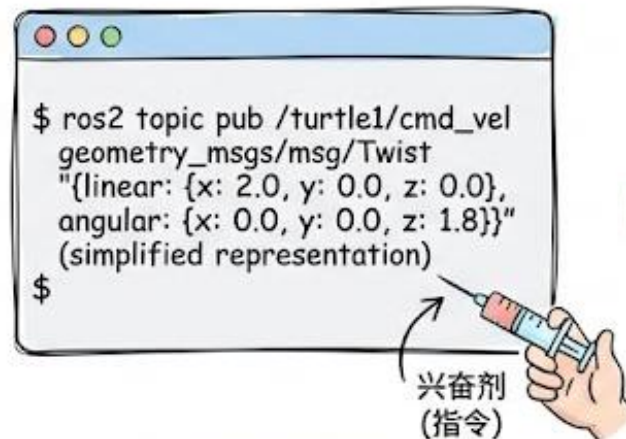
ROS2 医生技能升级：用“起搏器”手动接管控制

第一步：准备“起搏器”（概念）



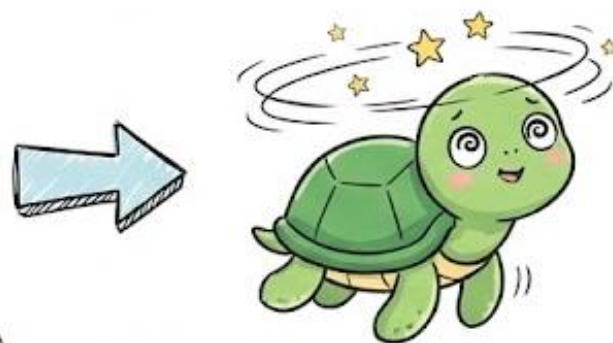
医生除了听诊，有时还需要用“起搏器”。

第二步：打一针“兴奋剂”（操作）



这条命令稍微有点长，就像是在给机器人打一针兴奋剂。回车!

第三步：见证“奇迹”（结果）



看，海龟开始转圈了。这一刻，你通过命令行强行接管了控制权。

在工程调试中，这是验证电机好坏的最快方法。

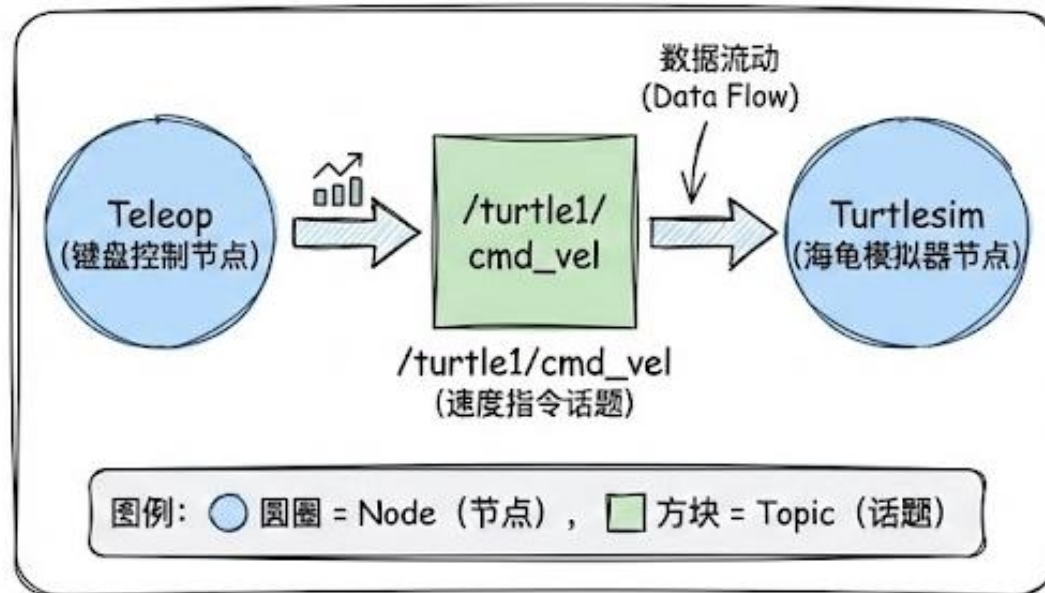
RQT Graph: 机器人的“思维导图”

终端文字太枯燥了! (Terminal Output)

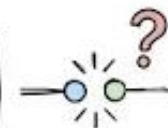
```
ros2@roboaster: ~$ ros2 topic list
/parameter_events
/parameter_events
/roscout
/turtle1/cmd_vel
/turtle1/cmd_vel
/turtle1/cmd_vel
/turtle1/rude_catents
/turtle1/test
/roscout
/turtle1/cmd_vel
/turtle1/cmd_vel//htep_assault_and
/turtle1/cmd_vel/thanko_and peorn
/turtle1/cmd_vel/snecoreos/pato
/turtle1/cmd_vel/locetator/tes
_set_wot
/turtle1/cmd_vel/jxento_de_la
/turtle1/cmd_vel/locetation_ontenis
/turtle1/cmd_vel//parameter_events
/turtle1/cmd_vel/gorececeser_sp_ekers
/turtle1/cmd_vel/doe_resmeces
/turtle1/cmd_vel/resrospes
/turtle1/cmd_vel/roscout
/turtle1/cmd_vel/rosc/parameter_events
/turtle1/cmd_vel/roscout
/turtle1/cmd_vel/turtlesim
root@roboaster: ~$
```

太难懂了!

RQT Graph: 一张图看清数据流! (直观、重要)



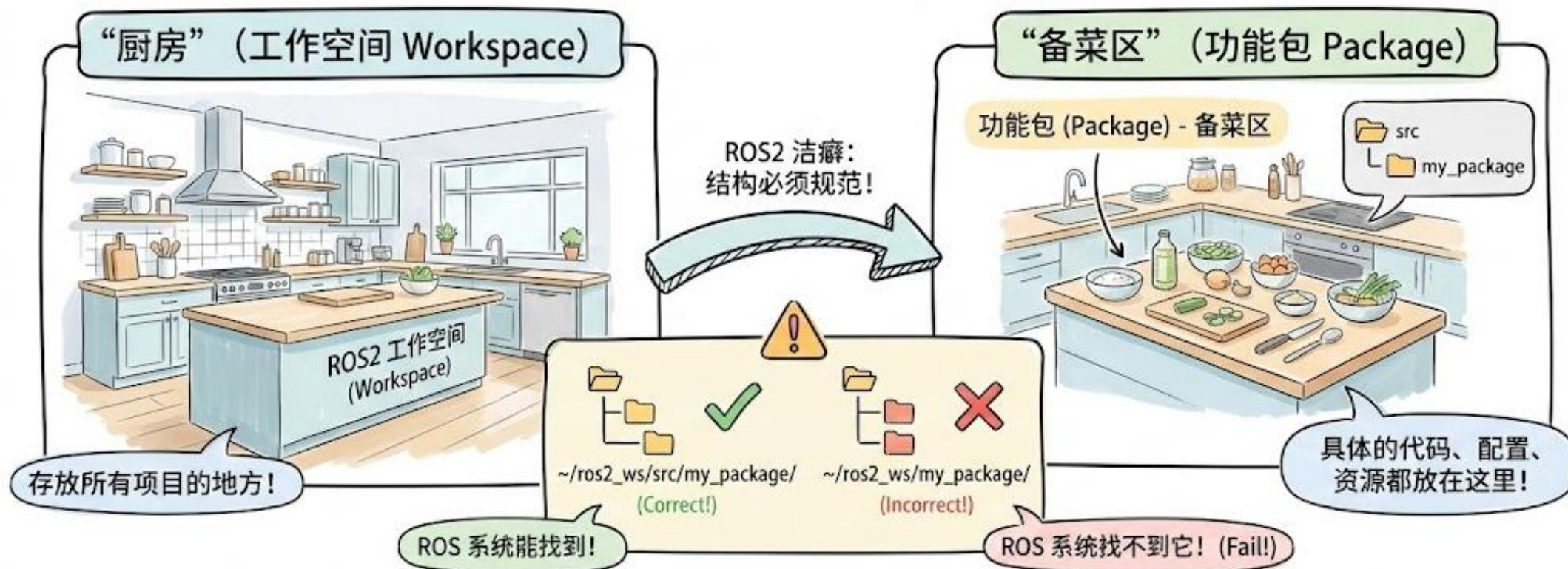
重要提示: 如果机器人不动了, 第一件事就是打开这张图, 看看是不是哪根线断了!





标准开发实战

ROS2 代码“厨房”：工作空间与功能包



开始写 Python 代码前，先建好你的‘厨房’和‘备菜区’！



第一个 ROS2 Node 代码解析：__init__ 的秘密

代码窗口 (Python Code)

```
class MyFirstNode(Node):  
    def __init__(self):  
        # 1. 告诉 ROS '我上线了'  
        super().__init__('my_first_node')  
        # 2. 建立发布者 (广播专栏)  
        self.pub = self.create_publisher(  
            Twist,  
            '/turtle1/cmd_vel',  
            10  
        )
```

这就像是在广播站注册了一个专栏，
准备好大声广播了！

图解类比 (Analogy)

第一件事：注册节点
(I'm online!)

我上线了!
(I'm online!)

ROS 系统注册处

第二件事：
创建发布者
(Publisher)

我要往这个
频道发消息!

专栏名称：
/turtle1/cmd_vel

广播站专栏注册

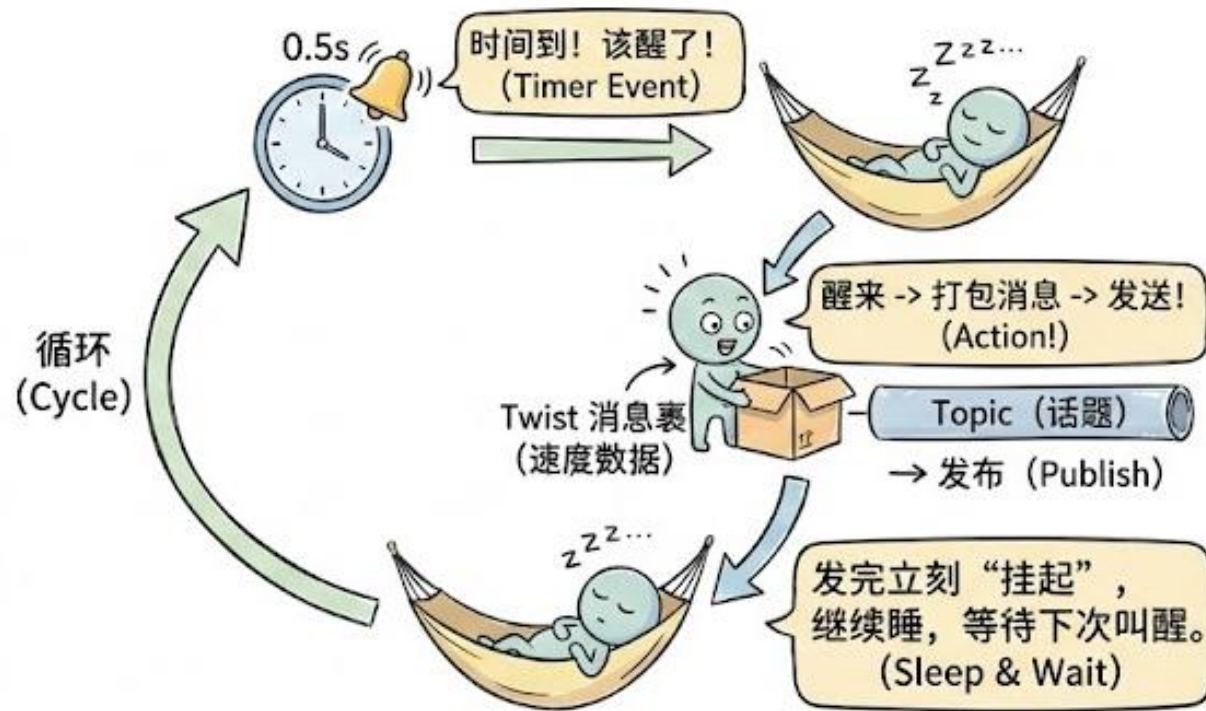
ROS 编程灵魂：定时器回调 (Timer Callback)

传统思维：While 循环（忙等待）



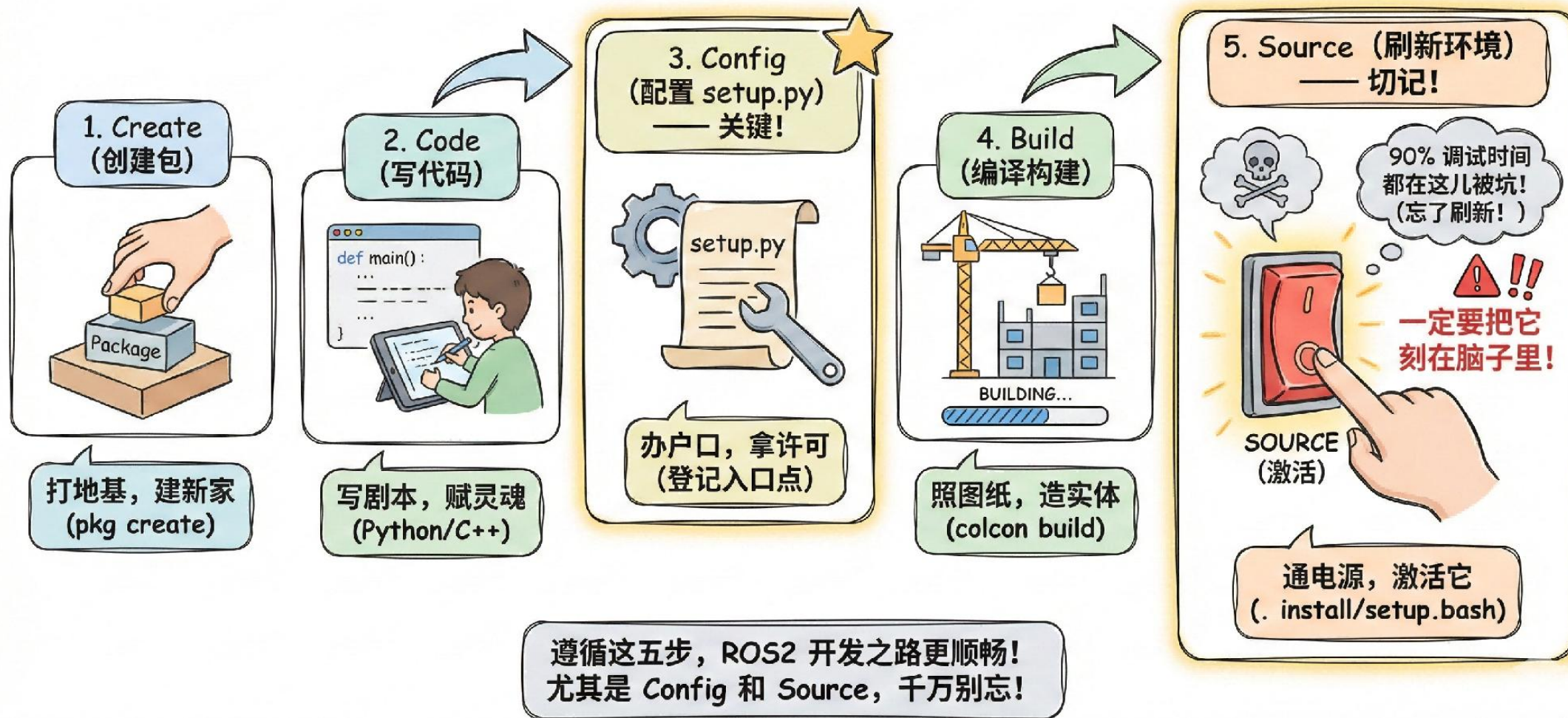
新手误区：效率低，占用 CPU!

ROS 之道：事件驱动 (Event-Driven) —— 高效!



这就是 ROS 的不同之处：不占用 CPU，只在需要时被“叫醒”工作!

十年经验总结：ROS2 开发“五步法”

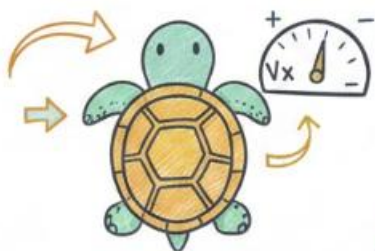




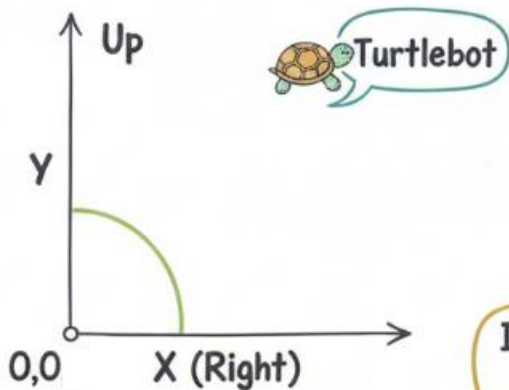
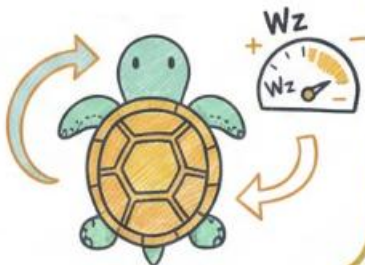
架构进阶

ROS Robot: Turtle Velocity Control

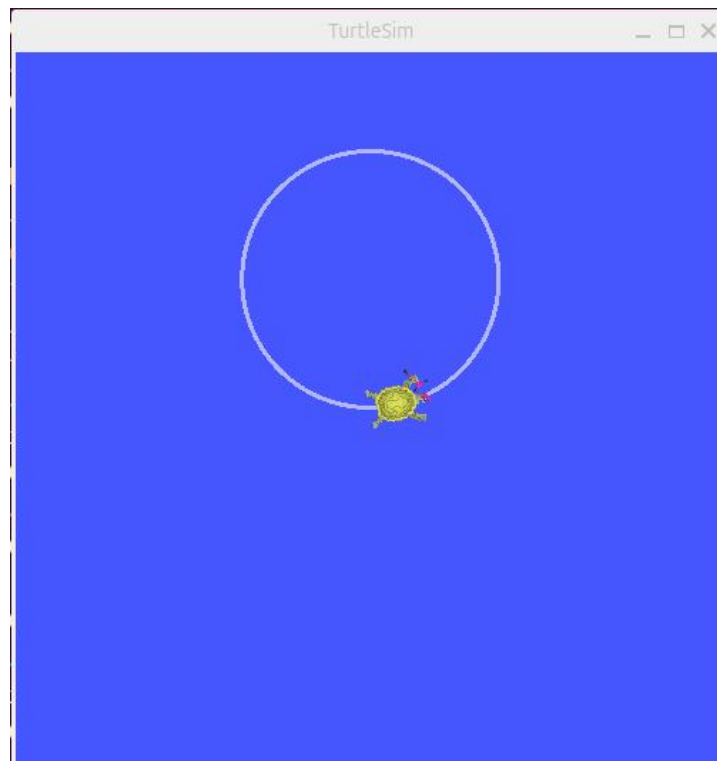
Linear Velocity



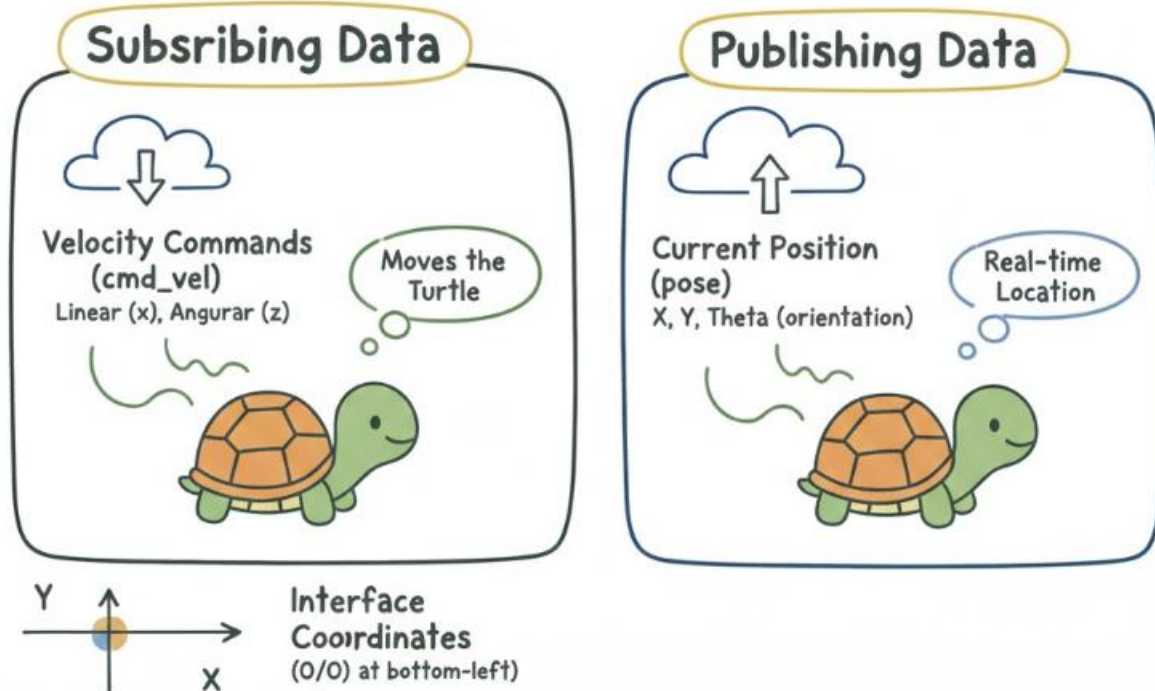
Angular Velocity



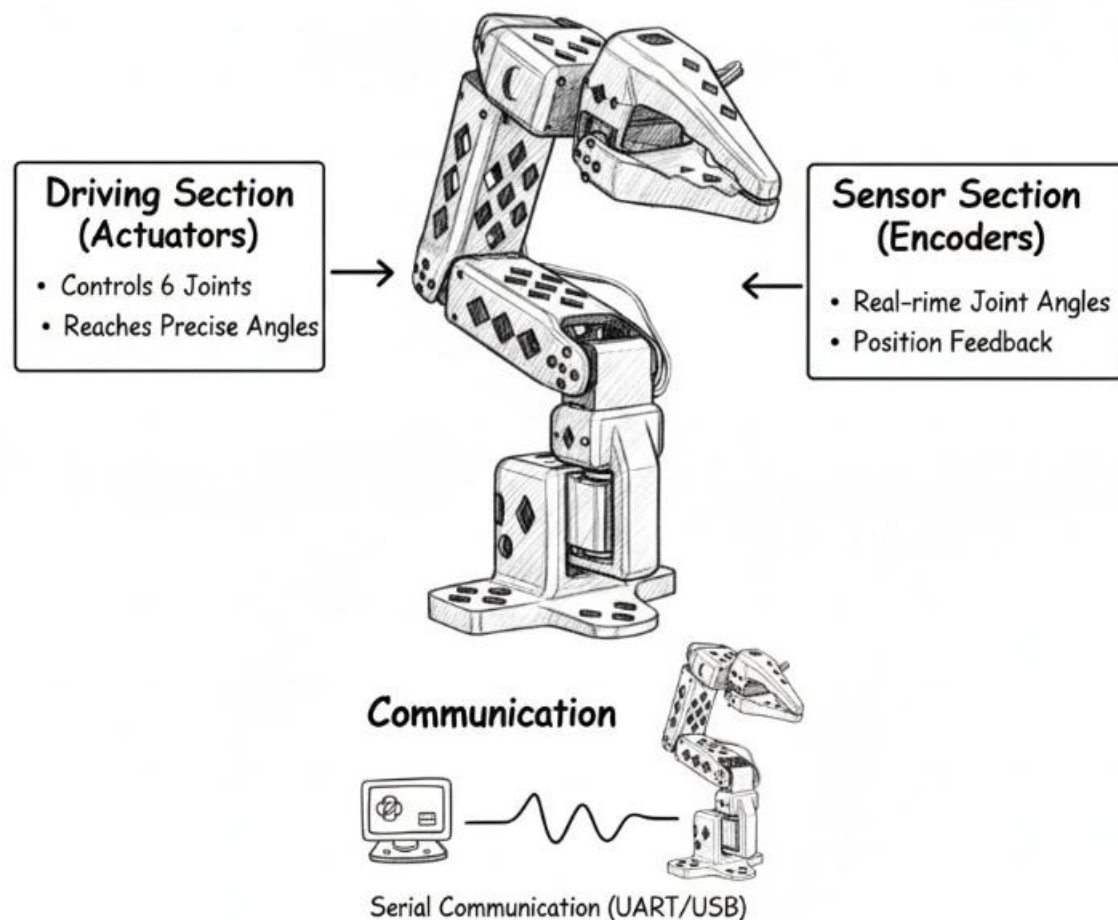
Interface: Bottom-Left (0,0),
X-axis (Right), Y-axis, Up



ROS Robot: Turtle Data Flow

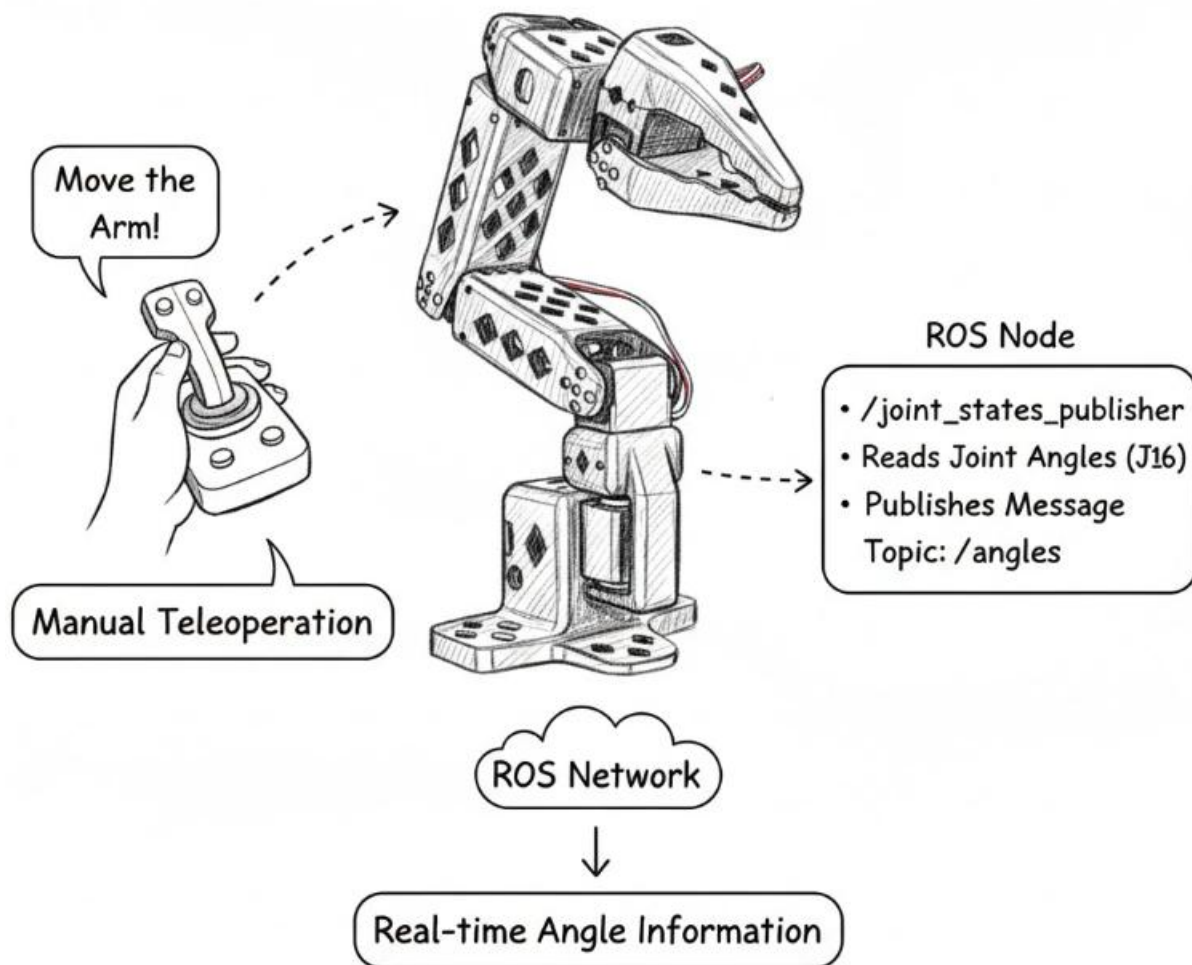


Six-Axis Robotic Arm: Control & Feedback



驱动器让机械臂动起来
传感器获取机械臂的关节角度

6-Axis Robotic Arm: Teleoperation & ROS Publishing



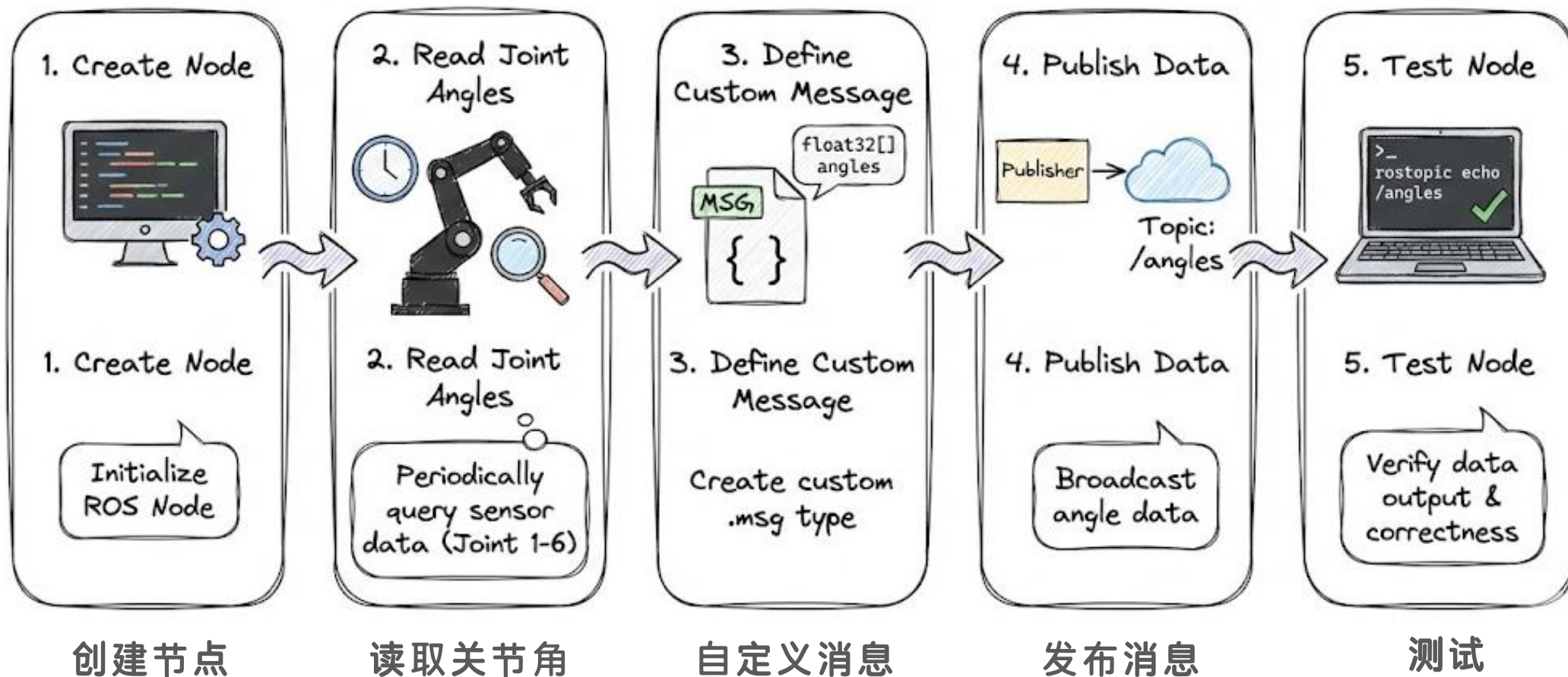
需求

为机械臂编写ROS的驱动节点实时获取机械臂的关节角度。

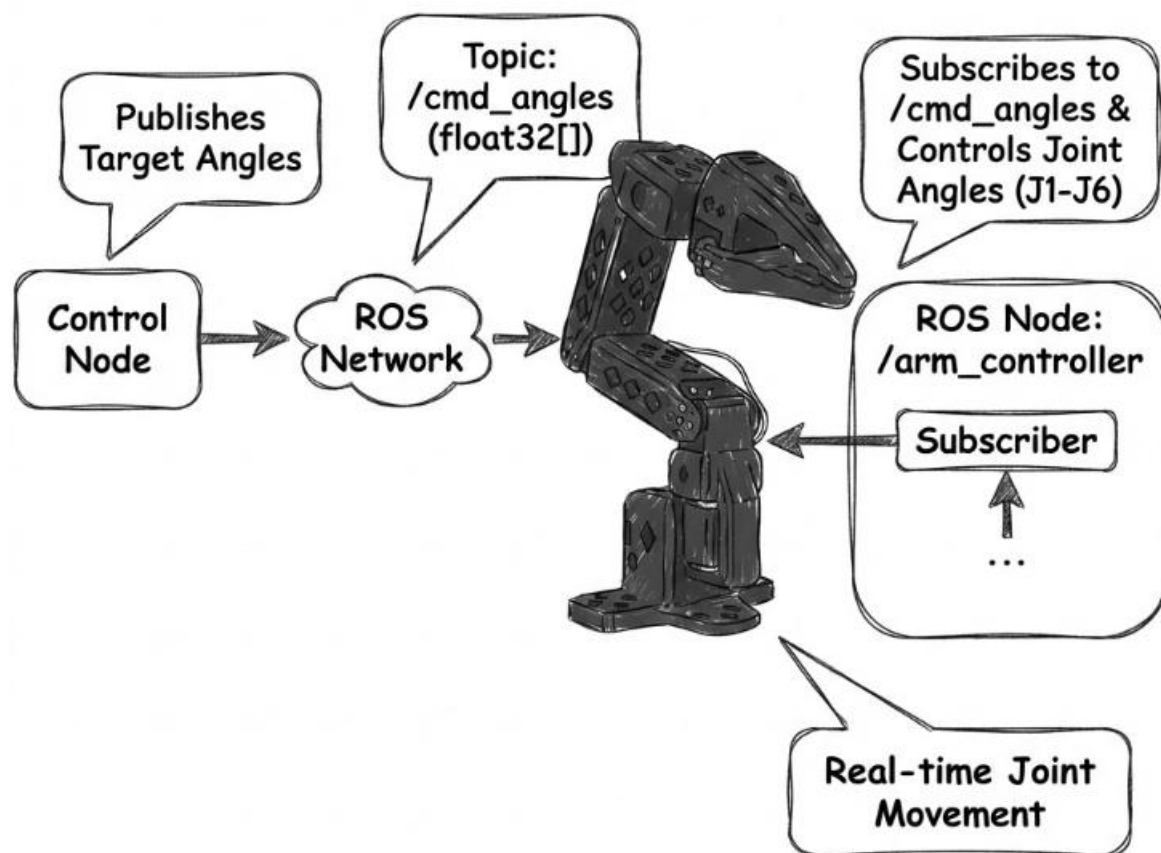
背景：

当摇动机械臂时，实时的将机械臂的关节发布出去

开发流程



Six-Axis Robotic Arm: Angle Control Subscription



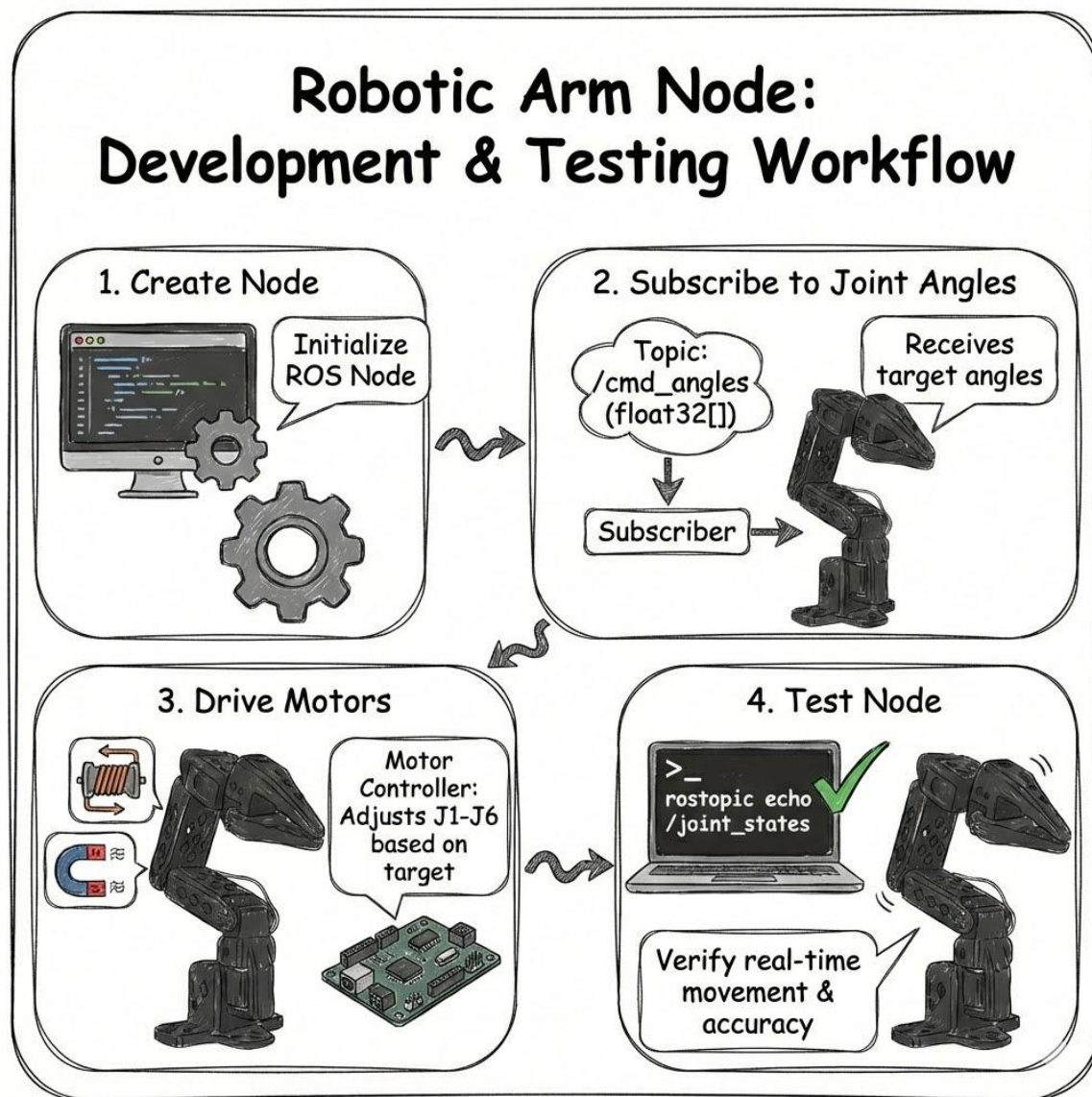
需求

为机械臂编写ROS的驱动节点实时驱动机械臂的关节运动到指定角度。

背景:

订阅其他节点发送的角度信息，基于角度信息让机械臂的关节运动到指定的角度。

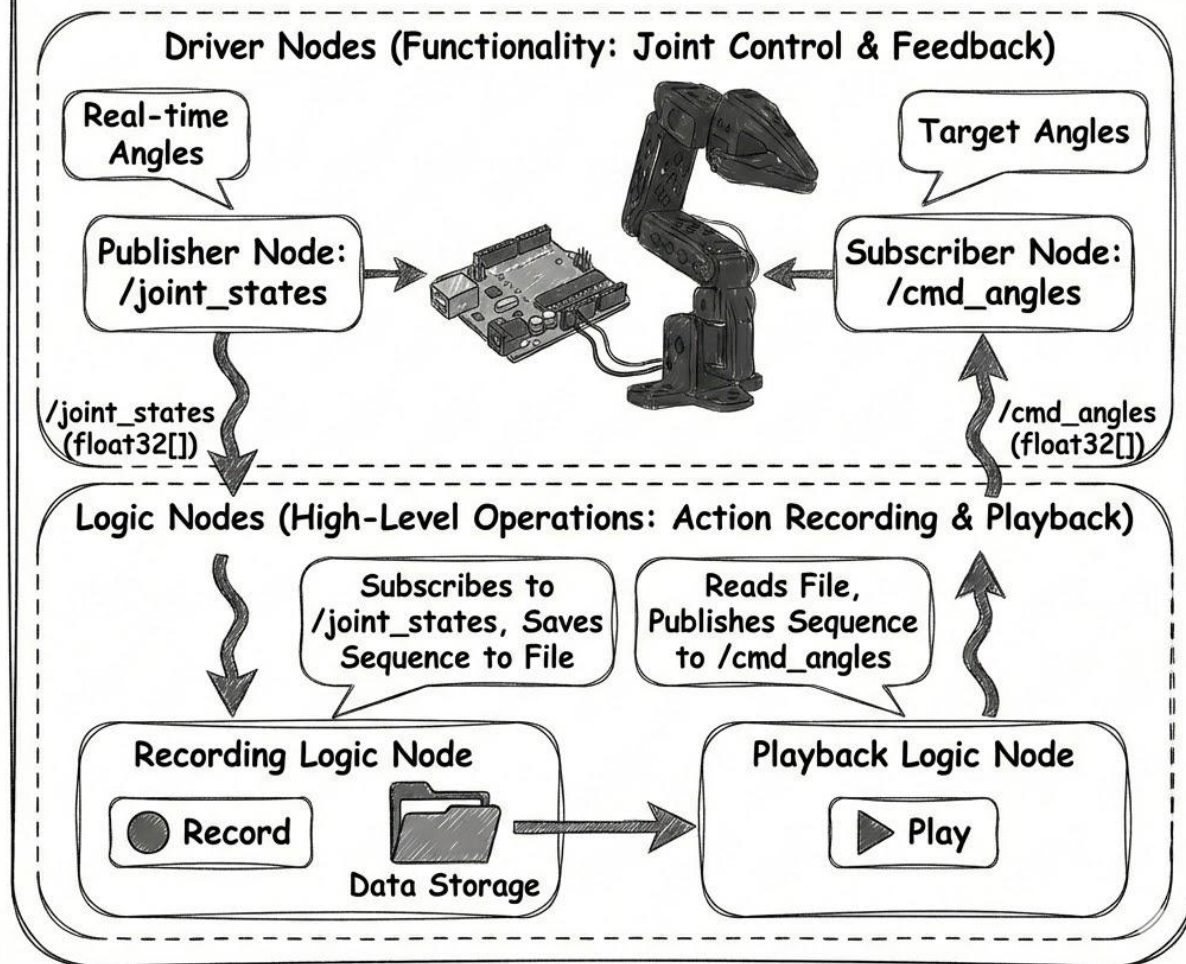
Robotic Arm Node: Development & Testing Workflow



开发流程

1. 创建节点
2. 创建订阅者
3. 驱动机械臂关节电机转动
4. 测试是否正常工作

Node Types: Driver vs. Logic Nodes & Action Recording

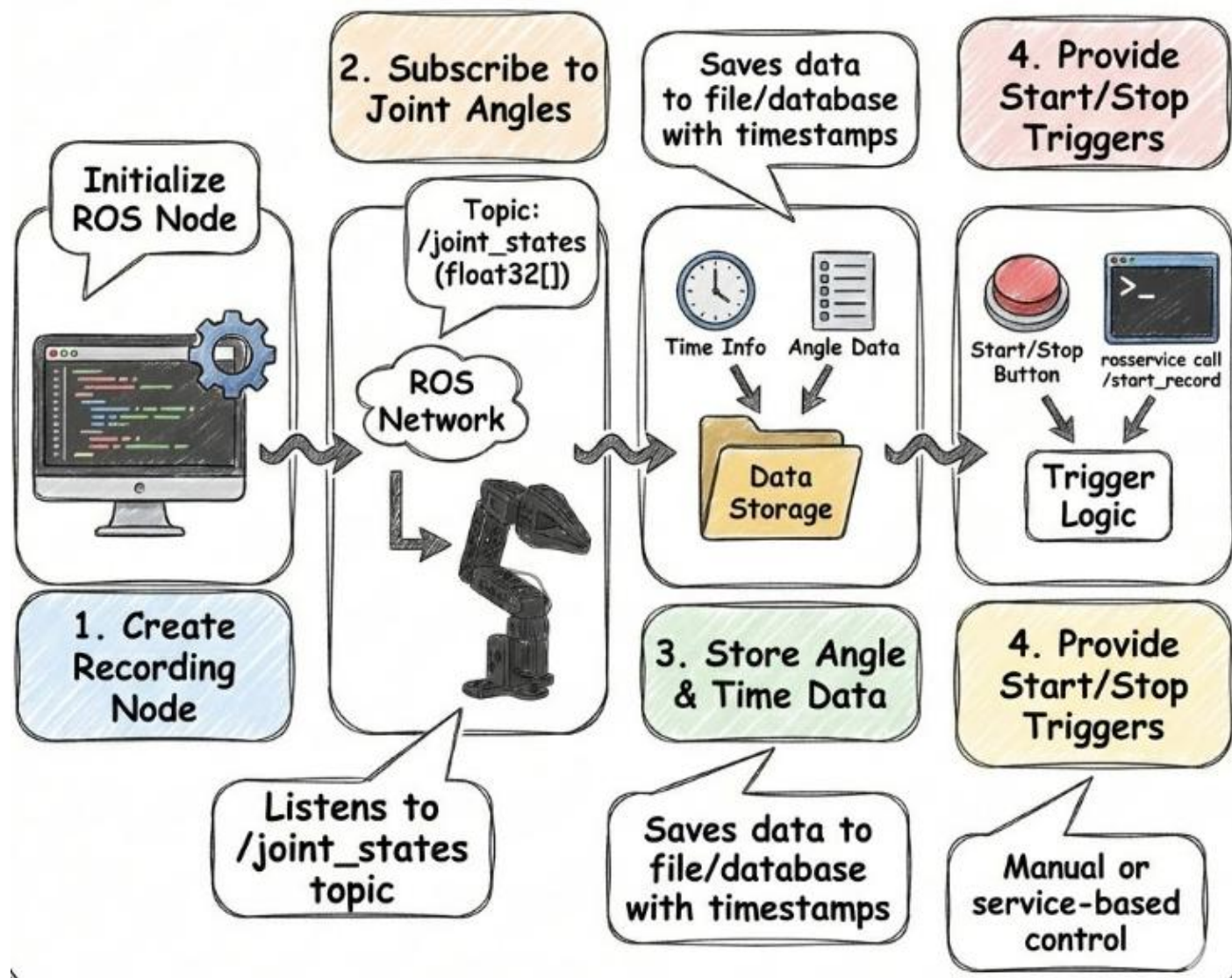


需求

实现机械臂动作的录制和
回放

背景:

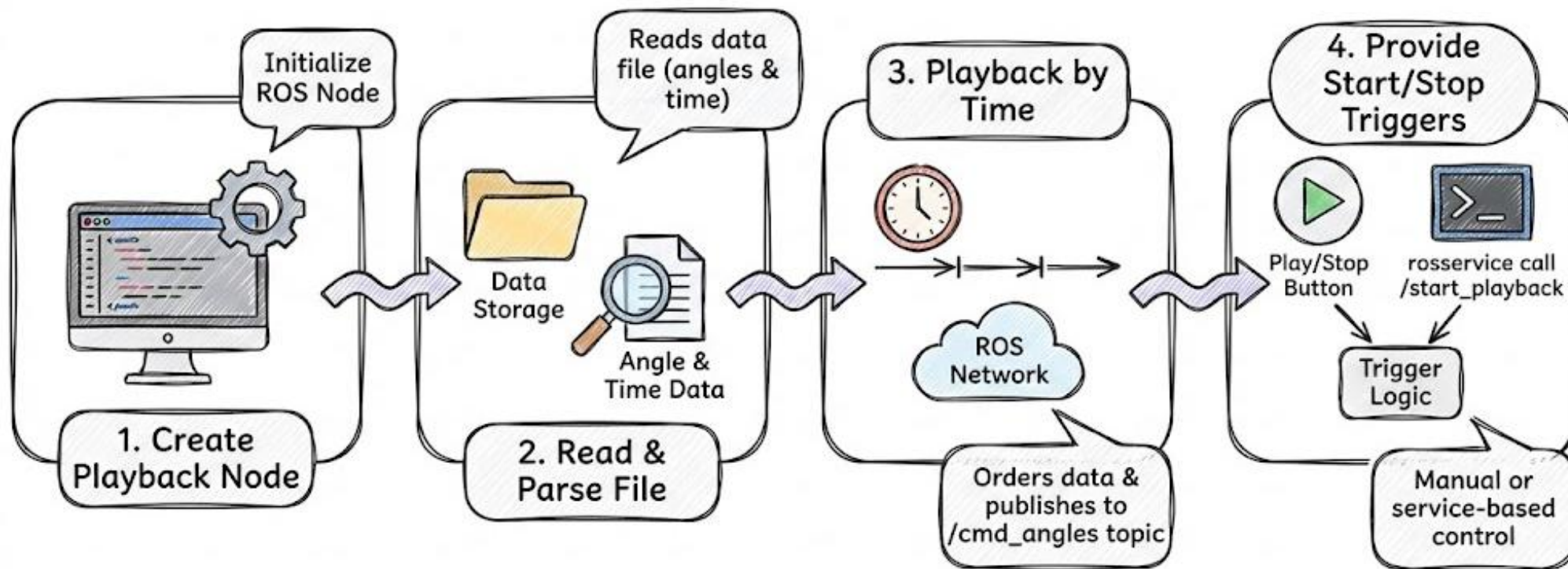
逻辑节点调用驱动节点实
现功能



录制流程

1. 创建节点
2. 订阅机械臂关节角度
3. 存储时间信息和角度信息
4. 提供启停操作接口

回放的开发流程

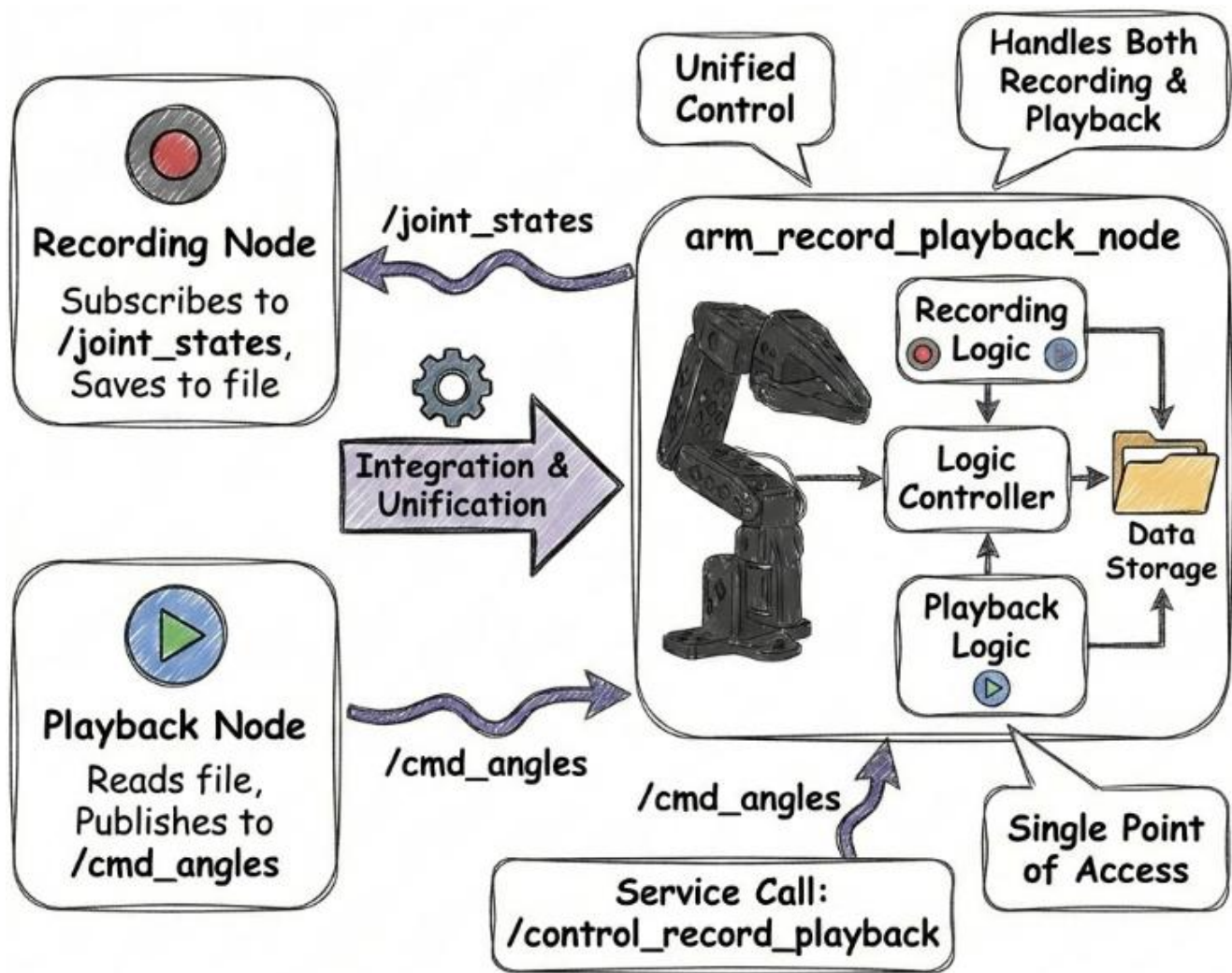


创建节点

读取和解析文件

播放文件

启停操作



功能整合

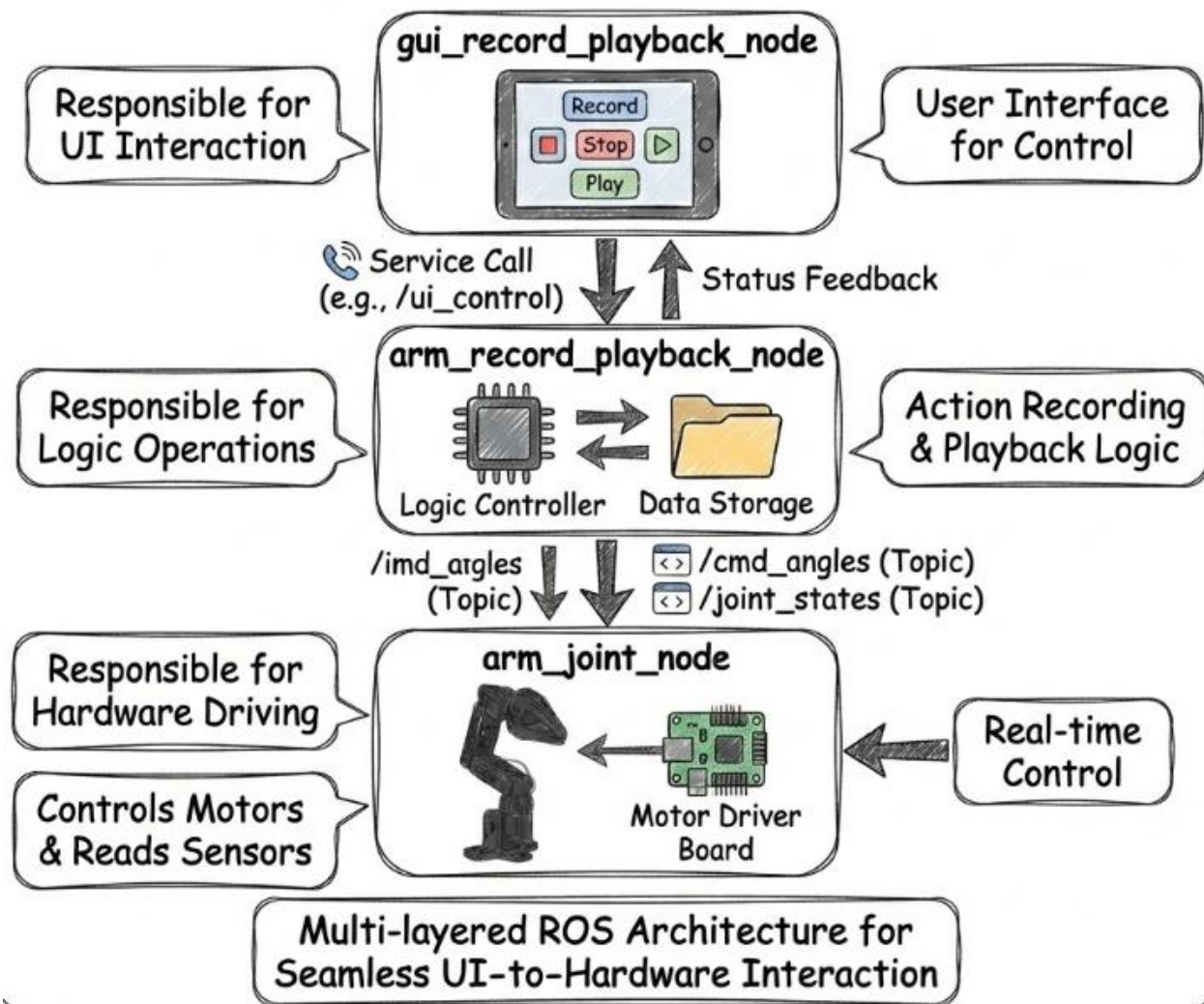
1. 录制和回放由一个节点提供
2. 提供录制启动停止操作
3. 提供回放启动停止操作
4. 提供状态查询接口
5. 录制回放功能不可以冲突

需求

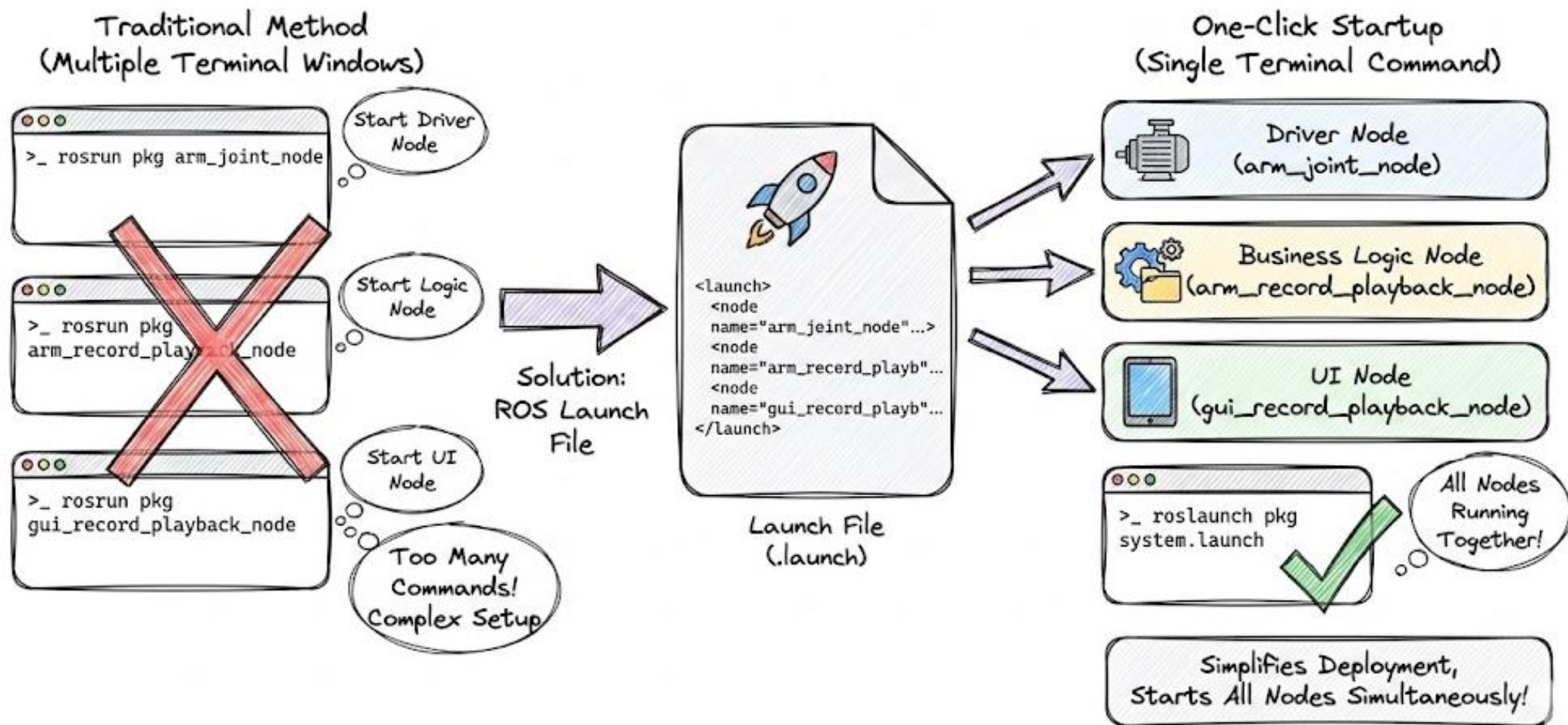
构建GUI交互方式进行录制和回放功能

背景：

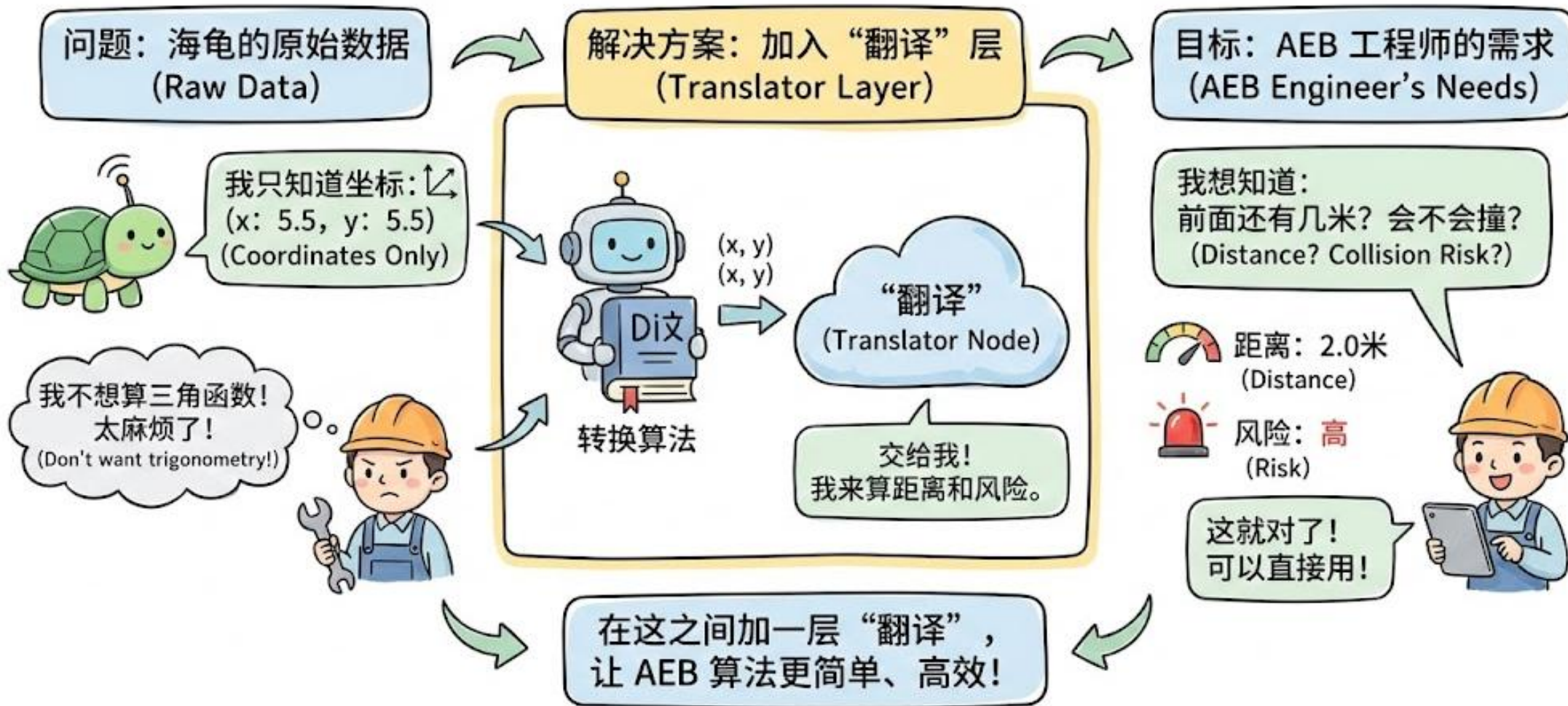
目前有机械臂驱动节点；
有有录制回放业务逻辑节点，
依赖驱动节点；提供UI交互
节点负责UI交互，依赖业务
逻辑节点。



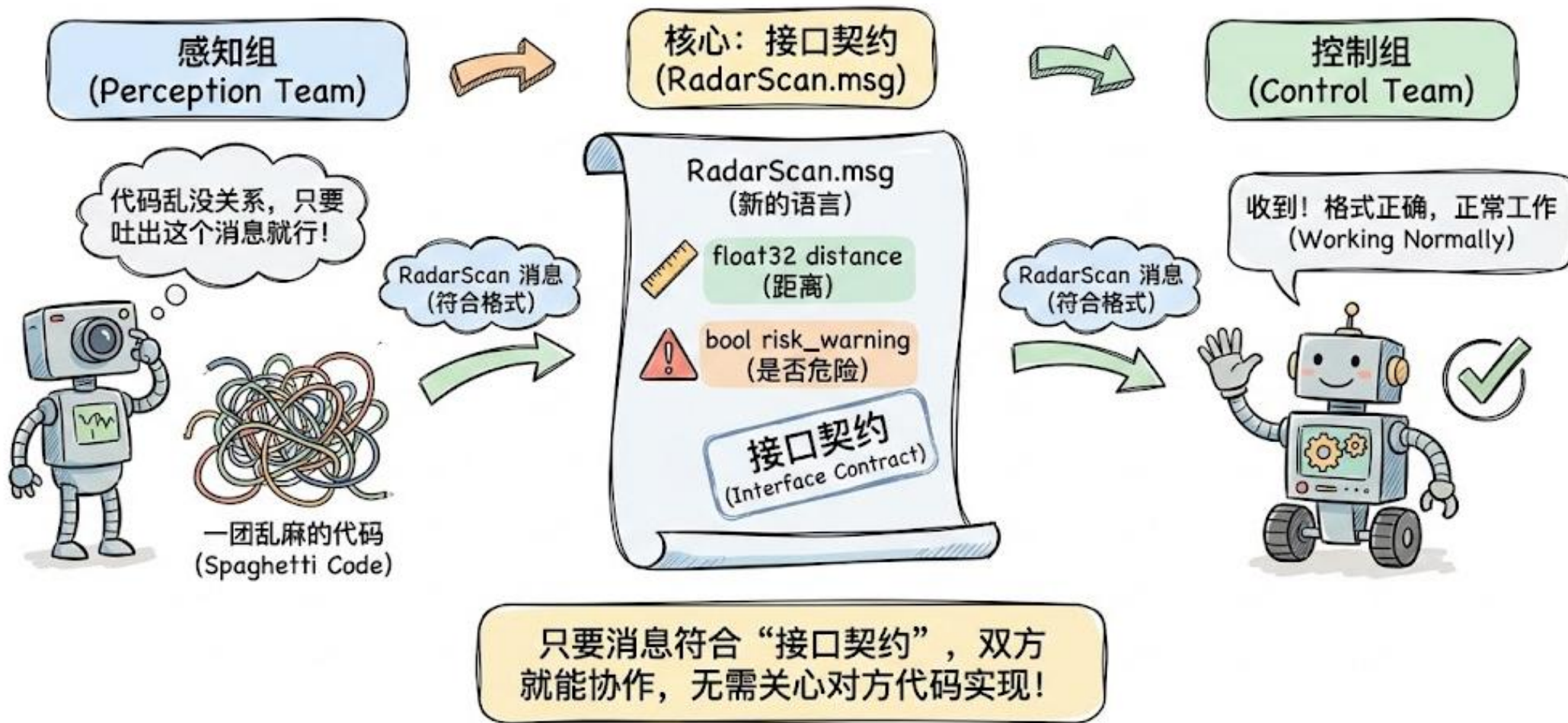
ROS launch一键启动配置



自动驾驶 AEB 系统：给海龟装上“翻译”



自定义 Msg: 接口契约的“翻译”语言



编译魔法 —— 让系统认识新语言



大家不要怕，这只是标准流程，照着抄就行。

验收时刻：创造机器人的“方言”

1. 验证命令

```
$ ros2 interface show  
  <your_msg_name>
```

编译完成后，输入它！

打印出了自定义
字段？恭喜！

```
$ custom message (...)  
...  
float32 distance  
bool risk_warning  
...  
$ □
```

2. 见证奇迹

3. 方言交流

RadarScan 消息
(我的方言！)

Perception
Node

收到！
我懂了！

Control
Node

现在，感知和控制节点可
以用这种方言交流了！

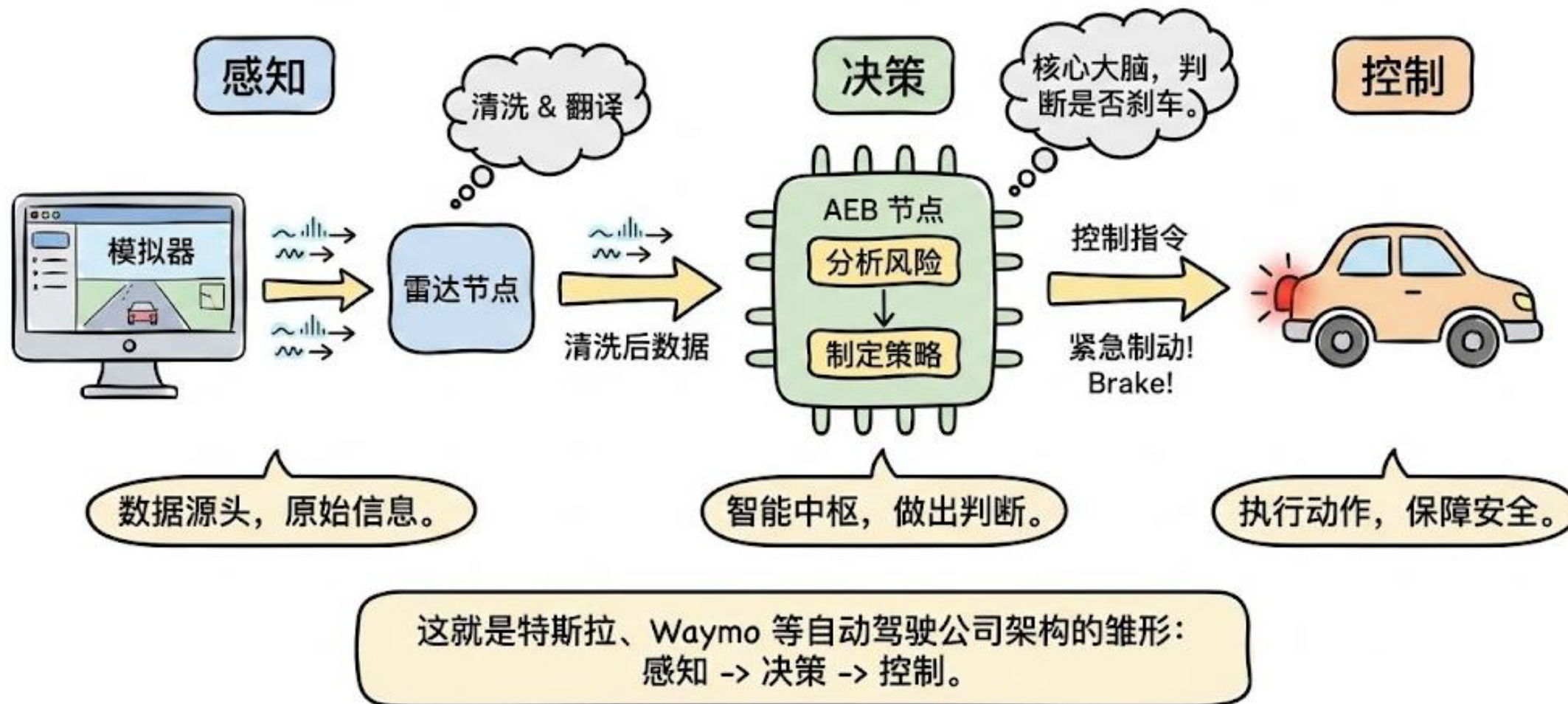


你刚刚创造了属于这台机器人的“方言”！



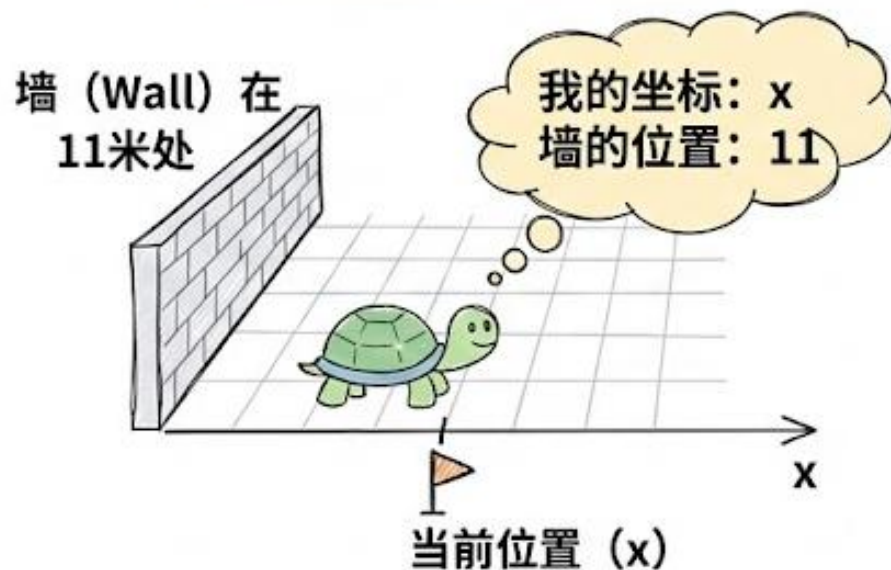
综合实战应用

自动驾驶系统架构总览 (System Architecture Overview)



编写感知逻辑：从数据中提取信息

1. 场景与原始数据



冰冷的坐标

2. 数据抽象与逻辑

$$\text{距离} = 11 - x$$

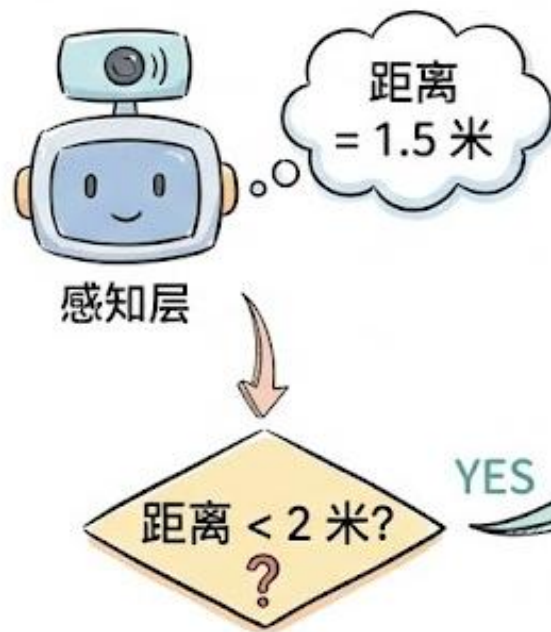
```
distance = 11.0 - msg.x
```

这行代码虽然简单，但它代表了“数据抽象”的过程。

核心概念：数据抽象

语义提取：从数据到判断

1. 数据与条件判断



2. 语义判断



这就是“语义提取”!

3. 黄金法则：单一职责原则



注意! 感知层不负责踩刹车!
一定要管住自己的手!

感知层验证：先测试，再前进

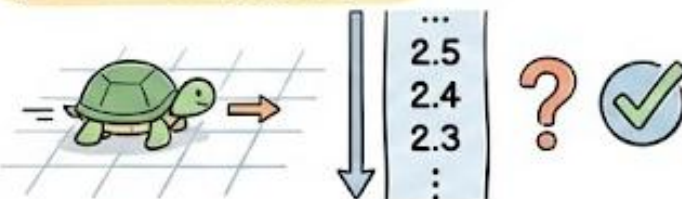
1. 运行与监听

```
>_  
ros2 run <perception_node>  
  
ros2 topic echo /radar_scan
```



2. 数据检查

检查 1: 距离变小?



检查 2: < 2米时, True 出现?



3. 确认与继续

确认无误!

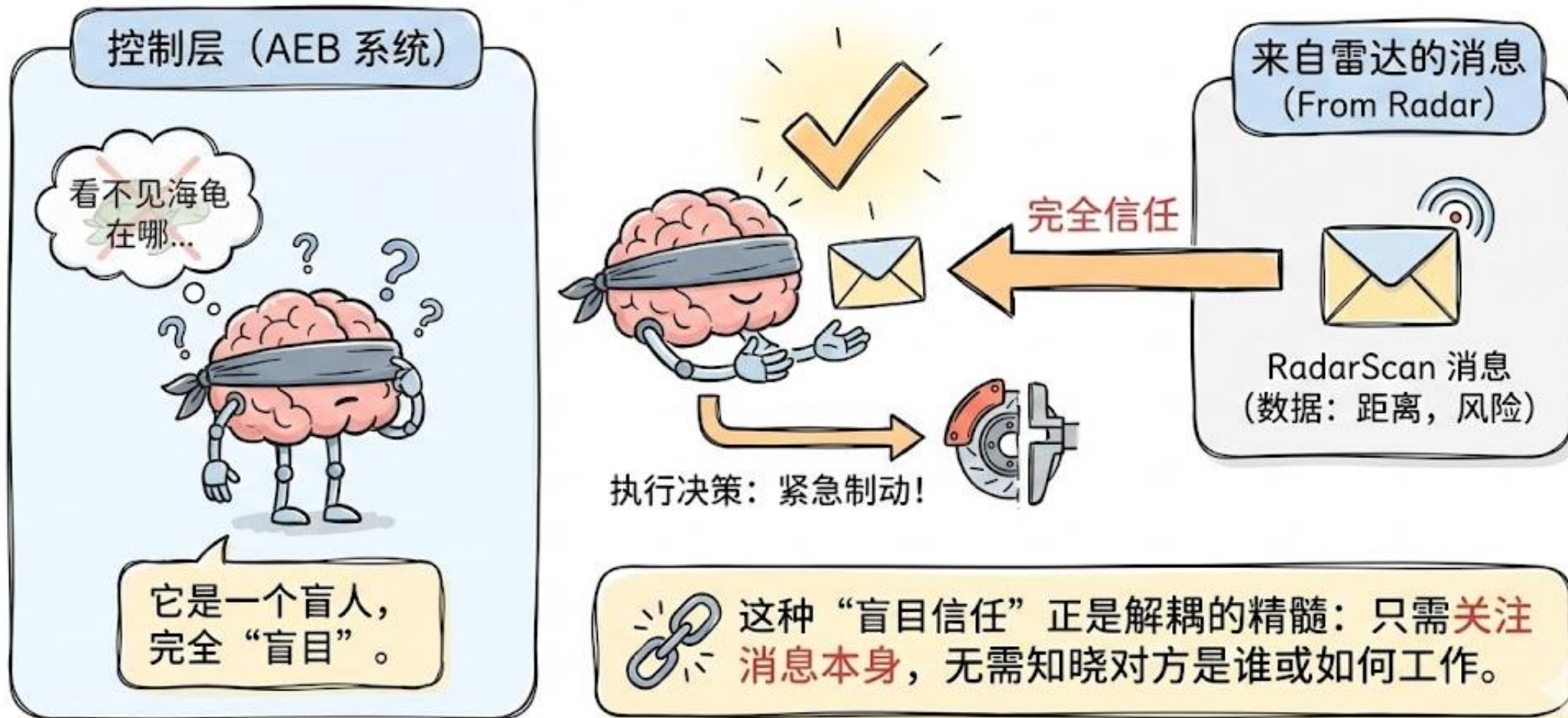


我们再继续!

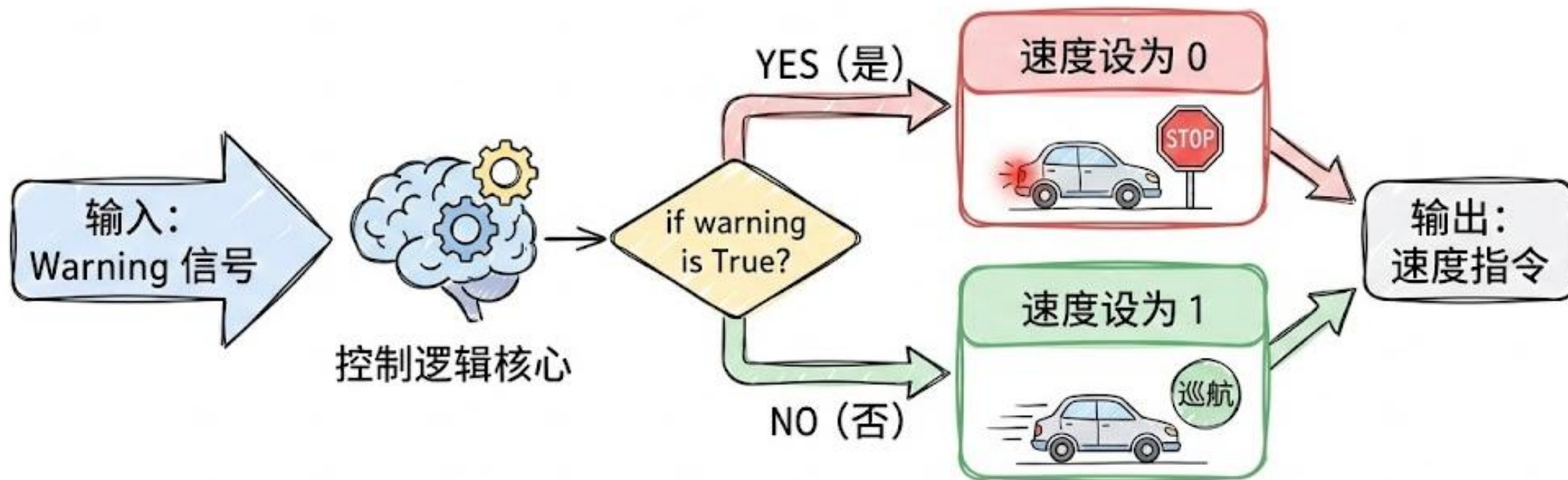


写完感知层，不要急着写下一层。先测试!

决策层设计：盲目信任与解耦精髓

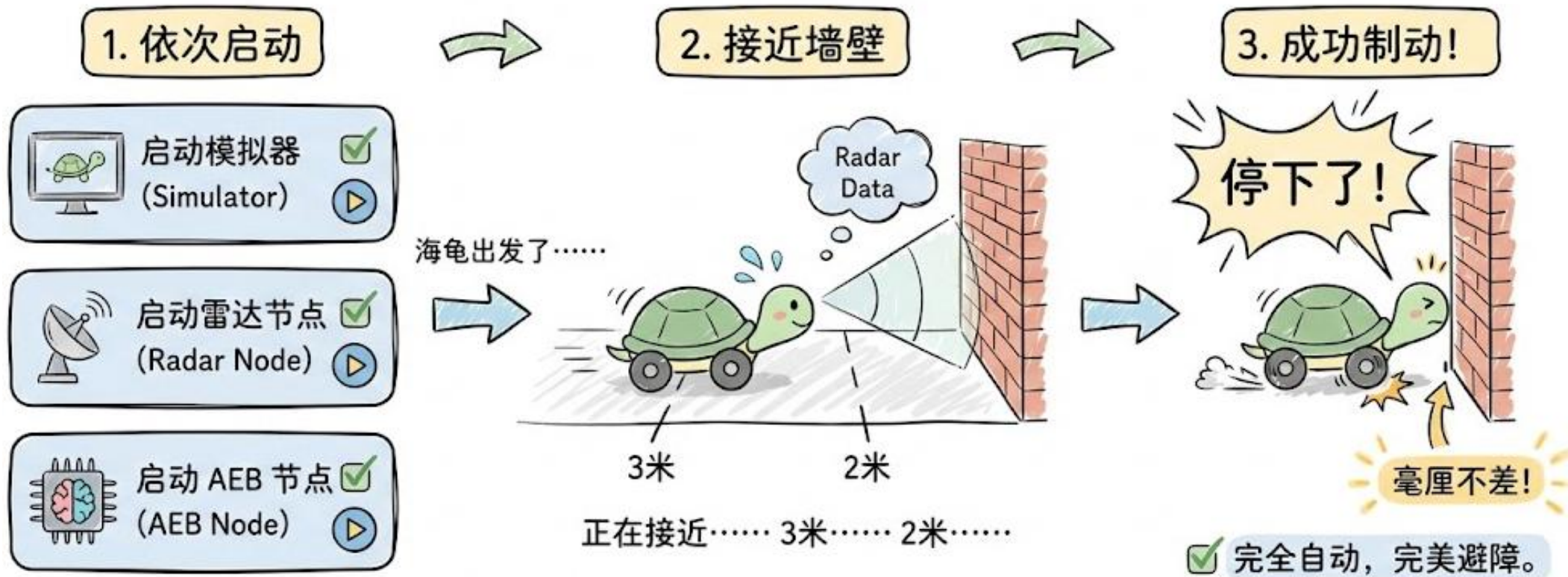


编写控制逻辑：系统的“大脑”



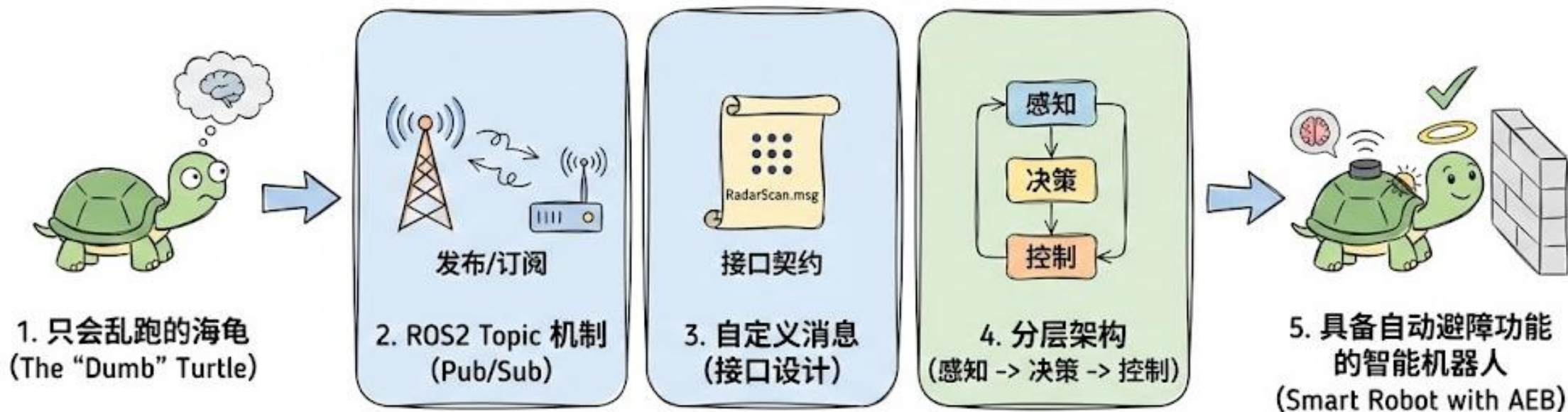
没有任何复杂的计算，只有最纯粹的逻辑判断！

最终联调 (The Demo)：见证奇迹的时刻!



🎉 🖐️ 掌声送给这只聪明的海龟! 🖐️ 🎉

总结与展望：从海龟到智能机器人



核心哲学：ROS 不仅仅是工具，更是一种**系统设计的哲学**。



传智教育旗下高端IT教育品牌